

Same code-review workload, same platform: the price list spans 267x from cheapest to most expensive.

Output prices for long-context models on one platform span \$0.15 to \$50 per million tokens today.

COST & LIMITS

OPENROUTER

EXAMPLE — CI VERIFICATION PENDING

THE DIFF

- budget the prompt: shave tokens, trim the system message
- + budget the model: the same month costs \$0.30 or \$80.00

WHY IT MATTERS

Above a certain volume the model you pick decides the bill, not how tightly you write the prompt.

WHO SHOULD CARE

Anyone running a high-volume automated workload: reviews, summaries, extraction, agent loops.

THE NUMBER

\$0.30 → \$80.00

the same 200-review month, cheapest against most expensive on today's list — 267x

- One review priced at 30,000 input and 2,000 output tokens, 200 reviews a month.
- The spread is on the price list, not on quality: this compares cost only.
- A cheaper model that needs two attempts is not cheaper — measure retries before switching.
- Cache reads and writes are priced separately and can reorder the ranking for repeated prefixes.
- Free tiers exist at \$0 but carry rate limits: they are not the same product.

RUN IT — 9 lines, CPU, offline, < 1 s

```
import json, pathlib
M = json.loads((pathlib.Path(__file__).parent / "prices.json").read_text())["models"]

IN_TOK, OUT_TOK, PER_MONTH = 30_000, 2_000, 200      # one code review, and how many a month

def monthly(m):
    return (IN_TOK * m["prompt"] + OUT_TOK * m["completion"]) * PER_MONTH

rows = sorted(M, key=monthly)
for m in rows:
    print(f"{m['id'][:42]} {m['context']//1000:>5}k ctx   ${monthly(m):8.2f} / month")

lo, hi = monthly(rows[0]), monthly(rows[-1])
print(f"\nsame workload, same platform: ${lo:.2f} vs ${hi:.2f} per month -> {hi/lo:.0f}x")
```

```
inception/mercury-2.5-preview          260k ctx   $    0.30 / month
z-ai/glm-5.3-flash                    1310k ctx   $    0.55 / month
ibm-granite/granite-4.2-8b             131k ctx   $    0.66 / month
meta/muse-spark-1.2-contributor        1048k ctx   $    0.68 / month
qwen/qwen3.8-flash                    1000k ctx   $    1.09 / month
deepseek/deepseek-v4-flash-vision-exp  1048k ctx   $    1.58 / month
tencent/hy4-preview                   1048k ctx   $    6.00 / month
anthropic/claude-fable-5.1            1000k ctx   $   80.00 / month

same workload, same platform: $0.30 vs $80.00 per month -> 267x
```

WATCH OUT

CAVEAT

This is a cost comparison, not a quality one, and prices move. The example vendors today's price list so the arithmetic stays reproducible — re-fetch before acting on it.

SOURCES & LINKS — every claim above traces to one of these

TYPE	WHAT	URL
Primary source	Original announcement / docs	openrouter.ai/api/v1/models
Runnable example	This exact script, re-run by CI	github.com/marcocm28/ai-daily-diff/tree/main/examples/2026-09-02-price-spread
Slides, video, brief	Everything from this episode	marcocm28.github.io/ai-daily-diff/2026-09-02/

Ollama now honours the sampling defaults declared inside the model file. Local outputs may shift.

Release v0.33.3 honours GGUF-defined default parameters, so a model runs with the settings its author shipped.

TOOLS & AGENTS

OLLAMA

EXAMPLE — CI VERIFICATION PENDING

THE DIFF

- the runner's own defaults, whatever the file declared
- + the defaults the model author shipped inside the GGUF

WHY IT MATTERS

If you relied on the runner's own defaults, this upgrade changes your output distribution.

WHO SHOULD CARE

Anyone running local models through Ollama in something that has to be reproducible.

THE NUMBER

12 → 4

tokens the model can pick at one step — same weights, only the defaults differ

- Total variation distance between the two distributions: 0.230, on a fixed logit vector.
- Top-token probability moves from 0.314 to 0.467: the model becomes more decisive.
- The example uses two plausible default sets, not Ollama's specific old values, which the release note does not state.
- If you already pin temperature and top_p explicitly in your calls, nothing changes for you.

RUN IT — 14 lines, numpy, CPU, offline, < 1 s

```
import numpy as np

# One fixed logit vector, so this is deterministic. It stands in for the model's output at a
# single step: nothing here depends on which model, only on the sampling parameters applied.
logits = np.array([4.1, 3.9, 3.4, 2.8, 2.7, 2.1, 1.6, 1.2, 0.9, 0.4, -0.6, -1.8])

def distribution(temperature, top_p):
    p = np.exp((logits - logits.max()) / temperature); p /= p.sum()
    order = np.argsort(-p)
    keep = order[:np.searchsorted(np.cumsum(p[order]), top_p) + 1]
    q = np.zeros_like(p); q[keep] = p[keep]
    return q / q.sum()

for label, temp, top_p in (("generic fallback", 1.0, 1.00), ("declared in the file", 0.6, 0.90)):
    q = distribution(temp, top_p)
    print(f"{label:22} temp={temp:.1f} top_p={top_p:.2f} "
          f"top token {q.max():.3f} tokens reachable {int((q > 0).sum())}/{q.size}")

a, b = distribution(1.0, 1.00), distribution(0.6, 0.90)
print(f"\ntotal variation distance between the two: {0.5 * np.abs(a - b).sum():.3f}")
print("same logits, same model weights - only the defaults differ")
```

```
generic fallback      temp=1.0 top_p=1.00   top token 0.314   tokens reachable 12/12
declared in the file  temp=0.6 top_p=0.90   top token 0.467   tokens reachable 4/12

total variation distance between the two: 0.230
same logits, same model weights - only the defaults differ
```

WATCH OUT

CAVEAT

We are not claiming what Ollama's previous defaults were: the release note says defaults are now honoured, and our example quantifies why that class of change matters. Check your own before and after.

SOURCES & LINKS — every claim above traces to one of these

TYPE	WHAT	URL
Primary source	Original announcement / docs	github.com/ollama/ollama/releases
Runnable example	This exact script, re-run by CI	github.com/marcocm28/ai-daily-diff/tree/main/examples/2026-09-02-gguf-defaults
Slides, video, brief	Everything from this episode	marcocm28.github.io/ai-daily-diff/2026-09-02/